Week 8

Aim

To attend the lab and get feedback on my lab work, once this feedback is received my aim is too than document the feedback.

Feedback

- Add comments to my code
- Include reflections of the code, learning reflections, criticisms of my own work
- Complete challenges for each lab by going beyond the work for higher grade
- Show him that I am thinking about the work deeply
- Redo my Lab4 work final challenge and use a brute force instead of doing a mathematical worm propagation.
- Include lecture notes
- Expand on my wapiti work

Fix

<u>Comment to my code</u>

```
Click to add a breakpoint and modules
import sys
!{sys.executable} -m pip install cryptography
import socket
import os
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import pickle
# Loads the receivers keyand reads it and creates a message to be sent
with open("public_key.pem", "rb") as f:
    public_key = serialization.load_pem_public_key(f.read())
message = b"Hello from the secure sender! This is confidential."
# Random 32 byte aes key
aes_key = os.urandom(32)
# Random vector to ensure unique ciphertext
iv = os.urandom(16)
# Creayes an AES cipher object
cipher = Cipher(algorithms.AES(aes_key), modes.CFB(iv))
encryptor = cipher.encryptor()
# Encrypts the message
encrypted_message = encryptor.update(message) + encryptor.finalize()
# Encrypts the AES key using the receivers public key
encrypted_key = public_key.encrypt(
aes_key,
padding.OAEP(
mgf=padding.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(),
label=None
)
# Bundles to be transmitted easier using pickle
)
payload = pickle.dumps((encrypted_key, iv, encrypted_message))
# Creates a TCP socket and sends the payload
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("localhost", 65432))
    s.sendall(payload)
    print("✅ Encrypted message sent!")
```

Added comments throughout my entire code to represent what was going on.

Include reflections of the code, learning reflections, criticisms of my own work

Week 4

Aim

The aim of this seminar was to look more into malicious software, how it operates and how it can be detected. Through completing the workshop, my aim was to gain a better understanding of malware detection systems and how we can utilise this knowledge in the future to build applications that can protect users better.

Glossary

Anti-virus – A security program that is designed to bloc, detect and remove malware.

Hashing – Mathematical process that converts data into a fixed length string.

Comparison system – A method that compares two values for example comparing hashes against stored hash.

Malware connection – The communication channel used by malware to connect to a server and spread to other systems, this can include features such as infected file propagation.

Detection – The process of identifying malicious activity.

Signature – A unique pattern associated with a specific piece of malware.

Payload – Part of the malware that performs harmful action.

Worm – A self-replicating type of malware that spreads automatically across a network without requiring user intervention.

File integrity - A security feature that ensures that a file has not been tampered with.

Discussion Points

Section 1: Discussion points

- Why do security systems rely on file hashes instead of timestamps or file sizes?

This is because a hash is a unique value which represents the content of the file, therefore even if one bit of the file is changed, the entire hash value is then altered. This alteration allows for the users to be alerted that their file has been changed and tampered with and therefore the integrity of

Errors and Solutions

During the completion of the workshop, I ran into the challenge of understanding the countermeasure design challenge. Initially, I believed that I had to mathematically simulate the decrease in worm propagation however all that was required for that was one simple function which did not seem right. Additionally, the course worked asked to work in a team, I did team up with someone however we never managed to collaborate on the coursework together and therefore decided to complete the task alone. Eventually, during week 8 feedback, I asked and was recommended to create a brute force defence system which was what I was initially speculating on doing.

Key Takeaways

Malware is more than just code – Malware also consist of a wide variety of behavioural patterns which can be hard to identify with simple static code, with AI growing every day, the ability to combat this will also grow. Working with analysing static code also made me realise how sophisticated malware can become.

Network based threads spread faster than expected – During my brute force simulation and worm propagation coursework I realised how quickly an outbreak can spread without simple implementation of rate limiting. It highlighted to me how important network layering really is.

Detect and Monitor – From this workshop, I also took away how vital it is to keep monitoring and detection services on your system. Knowing what is being hosted on your system is so important and often just by simple updates to your network you can avoid attacks.

I added an aim and glossary of the key terms, alongside with this, when the labs would have discussions points, I would answer the questions. Additionally, I added an errors and solutions section which highlighted my struggles and how I overcame them as well as key takeaway of what was most important to me from the seminar.

Complete challenges for each lab by going beyond the work for higher grade

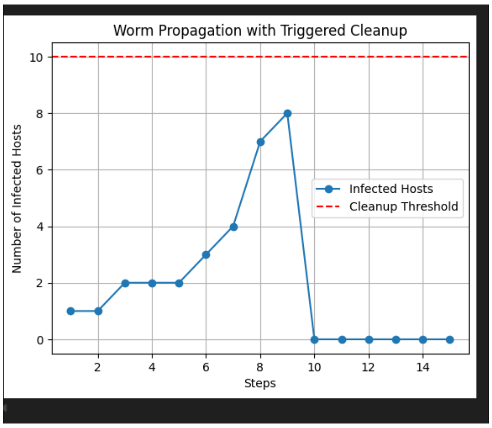
(Challenge) Section 6: Finding live hosts that are up

```python
import nmap
import ipaddress
#Performs a basic discovery scan across a subnet to identify devices on exist
def discover_hosts(network):
    nm = nmap.PortScanner()
    print(f"\nDiscovering devices on: {network}\n")

    try:
        #converts network object to a string
        nm.scan(hosts=str(network), arguments='-sn')
        #Identifies all hosts that responded with up state and than prints ou
        live_hosts = [host for host in nm.all_hosts() if nm[host].state() ==
        print(f"Devices found: {len(live_hosts)}\n")
        for host in live_hosts:
            print(f" {host}")
        return live_hosts

    except Exception as e:
        print(f"Error discovering network: {e}")
        return []


if __name__ == "__main__":
    network = ipaddress.ip_network("192.168.1.0/24", strict=False)

    discover_hosts(network)
```

Python

At the end of each of my lab ipynb files, there would be a ‘’(Challenge)’’ where in which I have submitted the extra challenges in which I have completed for the seminar.

<u>Redo my Lab4 work final challenge and use a brute force instead of doing a mathematical worm propagation.</u>



During Week 4, the seminar final challenge confused me because I was unable to understand how to complete the final project. Initially, I thought I needed to mathematically decrease the worm propagation however the execution function for this was far to simple and therefore I felt as though I needed to do something more complex. Therefore, I created a small brute force attack with a limit that stops when it has been reached. I kept my initial worm propagation since it showed my initial attempt at the challenge.

<u>Include lecture notes</u>

Created a folder that consists of my paper taken notes for the upcoming quiz/exam in a folder called ‘Lecture_Notes’.

<u>Expand on my wapiti work</u>

For my week 5 seminar work, I wanted to expand more than just a simple wapiti search since my aim was to complete one challenge for each lab to push my grade to a higher boundary. Therefore, I decided to utilise one of my own personal websites which goldsmiths' servers host for my dynamic web app module where in which I than used wapiti to perform a web vulnerability test on my own website which has an SQL database and many input forms. This report is then attached to my lab_5 under the name gold_report.pdf